Lecture 23: MIPS Arithmetic Instructions (add, addi, sub, mul, div, mfhi, mflo & move)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand how arithmetic is performed in MIPS.
- Use `add`, `addi`, `sub`, `mul`, and `div`.
- Understand immediate values.
- Learn the purpose of the HI and LO registers.
- Use `mfhi` and `mflo` after division.
- Use the `move` pseudo-instruction.
- Write complete arithmetic programs in MARS.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn the most important MIPS arithmetic instructions, including `add`, `addi`, `sub`, `mul`, `div`, `mfhi`, `mflo`, and `move`, with practical examples in the MARS simulator."**

In the previous lecture, we learned how to declare variables, arrays, and strings using assembler directives.

Now it's time to perform calculations.

Just like C++ has operators such as +, -, *, and /, MIPS Assembly Language provides arithmetic instructions that allow the CPU to perform mathematical operations.

In today's lecture, we'll learn the most important arithmetic instructions used in almost every MIPS program, including addition, subtraction, multiplication, division, and data movement between registers.

By the end of this lecture, you'll be able to write complete arithmetic programs in MARS.

Let's begin.

---

2. Arithmetic in MIPS

Unlike high-level languages, MIPS arithmetic instructions **operate on registers**.

Example in C++:

sum = a + b;

Equivalent in MIPS:

add $t0, $t1, $t2

Meaning:

$t0 = $t1 + $t2

Before performing arithmetic, the operands must already be in registers.

---

3. The add Instruction

Syntax

add destination, source1, source2

Example

add $t0, $t1, $t2

Meaning:

$t0 = $t1 + $t2

Example Program

```
.text
.globl main

main:

    li $t1, 20
    li $t2, 15

    add $t0, $t1, $t2

    li $v0, 10
    syscall
```

After execution:

$t0 = 35

---

4. The `addi` Instruction

Sometimes we add a constant instead of another register.

Syntax

addi destination, source, immediate

Example

addi $t0, $t1, 10

Meaning:

$t0 = $t1 + 10

Example Program

li $t1, 25

addi $t0, $t1, 5

Result:

$t0 = 30

---

5. Register vs Immediate

Register Instruction	Immediate Instruction
Uses two registers	Uses one register and one constant
`add`	`addi`
Faster access to register values	Convenient for constant values

---

6. The `sub` Instruction

Used to subtract one register from another.

Syntax

sub destination, source1, source2

Example

sub $t0, $t1, $t2

Meaning:

$t0 = $t1 - $t2

Example Program

li $t1, 50

li $t2, 20

sub $t0, $t1, $t2

Result:

$t0 = 30

---

7. The `mul` Instruction

Used to multiply two numbers.

Syntax

mul destination, source1, source2

Example

mul $t0, $t1, $t2

Meaning:

$t0 = $t1 × $t2

Example Program

li $t1, 8

li $t2, 6

mul $t0, $t1, $t2

Result:

$t0 = 48

> **Note for Students:** Some MIPS simulators implement `mul` as a pseudo-instruction. The underlying multiplication hardware uses the `mult` instruction and the HI/LO registers, which we'll discuss next.

---

8. Multiplication Using `mult`

Syntax

mult $t1, $t2

Unlike `mul`, `mult` does **not** store the result in a general-purpose register.

Instead:

- Lower 32 bits → **LO**
- Upper 32 bits → **HI**

Example:

li $t1, 7

li $t2, 8

mult $t1, $t2

mflo $t0

Result:

$t0 = 56

---

9. The `div` Instruction

Syntax

div source1, source2

Division stores its results in two special registers:

- **LO** → Quotient
- **HI** → Remainder

Example:

li $t1, 25

li $t2, 4

div $t1, $t2

Result:

Quotient = 6

Remainder = 1

---

10. `mflo` Instruction

`mflo` means:

Move From LO

Example:

div $t1, $t2

mflo $t0

Now:

$t0 = Quotient

---

11. `mfhi` Instruction

`mfhi` means:

Move From HI

Example:

div $t1, $t2

mfhi $t3

Now:

$t3 = Remainder

---

12. Complete Division Program

.text

.globl main

main:

    li $t1, 25

    li $t2, 4

```
div $t1, $t2

mflo $t0
mfhi $t3

li $v0, 10
syscall
```

After execution:

$t0 = 6

$t3 = 1

---

13. The `move` Pseudo-Instruction

Used to copy data from one register to another.

Syntax

move destination, source

Example:

move $t0, $t1

Meaning:

$t0 = $t1

Internally, the assembler expands it into an equivalent real instruction (such as using `addu` with `$zero`).

---

14. Example Program

li $t1, 100

move $t0, $t1

Result:

$t0 = 100

---

15. Complete Arithmetic Program

.text

.globl main

main:

    li $t1, 15

```
li $t2, 5

add $t3, $t1, $t2
sub $t4, $t1, $t2
mul $t5, $t1, $t2

div $t1, $t2

mflo $t6
mfhi $t7

li $v0, 10
syscall
```

After execution:

Register	Value
`$t3`	20
`$t4`	10
`$t5`	75

`$t6`	3 (Quotient)
`$t7`	0 (Remainder)

---

16. Real-Life Analogy

Imagine a calculator.

- `add` → Press **+**
- `sub` → Press **−**
- `mul` → Press **×**
- `div` → Press **÷**
- `move` → Copy a number from one memory slot to another
- HI and LO → Two special compartments where the calculator temporarily stores division and multiplication results

---

17. Common Mistakes

Students often:

❌ Write:

add $t0, 5, 10

✔ `add` only works with registers.

Correct:

li $t1, 5

li $t2, 10

add $t0, $t1, $t2

❌ Forget to retrieve the quotient after `div`.

✔ Use:

mflo $t0

❌ Forget the remainder.

✔ Use:

mfhi $t1

❌ Confuse `add` and `addi`.

✔ `add` uses two registers.

✔ `addi` uses one register and one immediate constant.

🎤 Lecture Summary

Today, we learned how to perform arithmetic operations in MIPS Assembly Language.

We explored the `add`, `addi`, `sub`, `mul`, `mult`, `div`, `mfhi`, `mflo`, and `move` instructions, and understood how the HI and LO registers are used during multiplication and division. We also wrote complete arithmetic programs in the MARS simulator and practiced solving common programming problems.

In the next lecture, we'll learn **MIPS Memory Access Instructions**, including `lw`, `sw`, `lb`, `sb`, and address calculation. We'll finally connect registers with the data stored in memory and learn how MIPS loads and stores variables.

Thank you for watching, and I'll see you in the next lecture.
